

JARINGAN KOMPUTER
MODUL 7 SOCKET PROGRAMMING

Nama : Syahril Arfian Almazril
Nim : 103032300013
Kelas : IT-47-04
Kode Asprak : STO

A. Teori

1. Apa yang dimaksud dengan socket dalam konteks network programming? Jelaskan peran socket sebagai antarmuka komunikasi antara aplikasi dan jaringan. Bagaimana socket memungkinkan komunikasi antara dua perangkat di jaringan?

Socket adalah sebuah endpoint yang digunakan dalam komunikasi dua arah antara perangkat yang terhubung dalam sebuah jaringan. Dalam konteks network programming, socket berfungsi sebagai antarmuka (interface) yang memungkinkan aplikasi untuk berkomunikasi dengan aplikasi lain, baik secara lokal maupun melalui jaringan internet menggunakan alamat IP. Socket dapat menggunakan dua protokol utama, yaitu TCP (Transmission Control Protocol) yang bersifat terhubung dan UDP (User Datagram Protocol) yang bersifat tanpa koneksi. Dengan socket, perangkat dapat saling bertukar data dengan lebih efisien, baik dalam skala lokal maupun global.

Untuk memungkinkan komunikasi antar perangkat dalam jaringan, socket mengandalkan kombinasi alamat IP dan nomor port yang unik untuk setiap perangkat yang terhubung. Contohnya, alamat IP 192.168.0.10 dengan port 80 menunjukkan bahwa perangkat tersebut menggunakan IP 192.168.0.10 dengan port 80. Pada protokol TCP, proses koneksi dilakukan melalui perintah seperti `bind()`, `listen()`, `accept()`, dan `connect()`, yang mengatur sambungan antar perangkat. Sebaliknya, pada UDP, tidak ada proses handshake, dan data langsung dikirim dalam bentuk paket tanpa koneksi yang terjalin terlebih dahulu. Setelah koneksi terbentuk, data dapat dipertukarkan menggunakan perintah `send()` dan `recv()` untuk pengiriman dan penerimaan data.

2. Jelaskan perbedaan antara TCP (Transmission Control Protocol) dan UDP (User Datagram Protocol) dalam konteks socket programming. Berikan contoh kasus penggunaan masing-masing protokol dalam aplikasi nyata

Aspek	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Koneksi	Berorientasi Koneksi (handshake: SYN, SYN-ACK, ACK)	Tanpa Koneksi (tidak Memerlukan handshake)
Keandalan	Menjamin pengiriman data (acknowledgement dan retransmisi)	tidak menjamin pengiriman (without acknowledgement)
Urutan Data	Data diurutkan dan bebas duplikat	data mungkin tidak terurut atau terduplikasi
Overhead	Lebih Tinggi (header besar, kontrol aliran, dll)	Rendah (header kecil, tidak ada kontrol kompleks)
Kecepatan	Lebih lambat karena proses verifikasi (handshake & acknowledgement, dll)	lebih cepat karena minim overhead
Penggunaan pada socket	<code>bind()</code> , <code>listen()</code> , <code>accept()</code> , <code>connect()</code> , <code>send()</code> , <code>recv()</code>	<code>bind()</code> , <code>sendto()</code> , <code>recvfrom()</code>

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

Contoh Penggunaan Protokol TCP

Biasanya Protokol TCP digunakan untuk penggunaan yang harus memastikan data tersebut utuh dan berhasil di dapatkan dan sinkron contoh penggunaan:

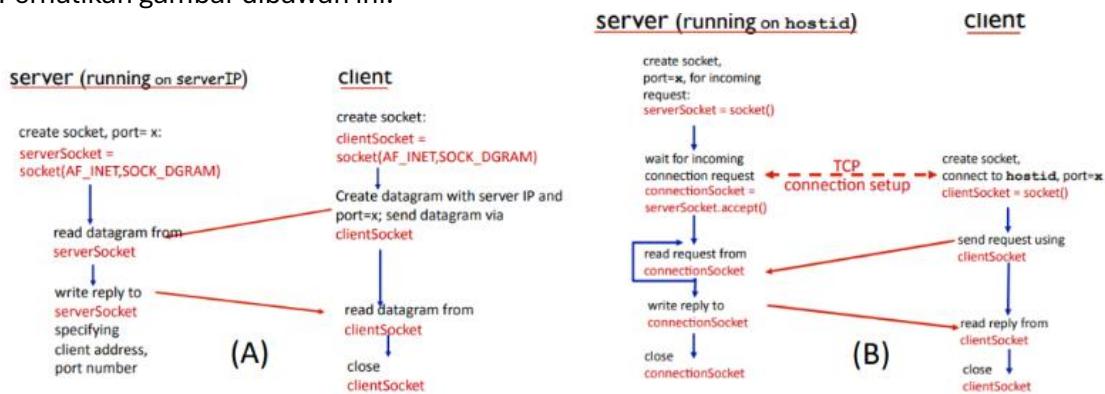
1. Web Browsing dengan HTTP/HTTPS yang utamanya mengirim dan menerima request, respon, accept dari client ke server dan server ke client dan server memberikan respon dari client dan mengirim informasi data dari website tersebut ke client dengan kepastian bahwa data dari server ke client itu utuh dan tidak ada data yang hilang dengan menggunakan mekanisme retransmission serta juga menggunakan mekanisme stuktur data yang sinkron / teratur.
2. Email (SMTP,POP3) dengan memastikan data dari client ke tujuan dengan kepastian data tersebut sampai ke tujuan tanpa ada data yang hilang
3. File Transferring (FTP) dengan mengirim file besar seperti video atau data dengan kepastian tidak terjadi error pada data tersebut

Contoh Penggunaan Protokol UDP

UDP digunakan untuk penggunaan aplikasi yang kecepatan dan efficiency, terutama responsivitas jaringan seperti:

1. Video Streaming (Youtube) secara konstant membutuhkan koneksi data yang cepat untuk tidak memiliki kendala / hambatan ketika memainkan video (streaming video).
2. VoIP (Voice Over IP) seperti Whatsapp yang harus membutuhkan data yang responsif dan tercapai ke tujuan.
3. Permainan Online (Multiplayer) yang digunakan untuk menentukan dimana pemain berada sekarang dan terus membutuhkan data yang yang cepat untuk tidak terjadi desync dan latency yang besar

3. Perhatikan gambar dibawah ini:



Jelaskan apa maksud dari kedua gambar diatas, serta sebutkan apa yang membuat kedua gambar tersebut berbeda!

Gambar A menggambarkan komunikasi menggunakan protokol UDP (User Datagram Protocol), yang merupakan protokol tanpa koneksi. Pada server UDP, pertama-tama socket dibuat menggunakan `socket(AF_INET, SOCK_DGRAM)` dengan nomor port tertentu, kemudian server menunggu dan membaca datagram yang diterima dari client. Setelah itu, server mengirimkan balasan ke client dengan menggunakan alamat IP dan nomor port yang telah ditentukan oleh client. Client UDP, di sisi lain, membuat socket dengan `socket(AF_INET, SOCK_DGRAM)` dan mengirimkan datagram ke server dengan alamat IP dan port yang telah ditentukan. Client kemudian menerima balasan dari server dalam bentuk datagram, sebelum menutup koneksi dengan `close()`.

Gambar B menggambarkan komunikasi dengan protokol TCP (Transmission Control Protocol), yang berfokus pada koneksi dan menjamin keandalan serta urutan data yang benar. Pada server TCP, socket dibuat untuk menerima permintaan dengan menggunakan `socket()` dan menunggu permintaan koneksi dengan `accept()`. Setelah menerima koneksi, server membaca dan memverifikasi request client, memastikan data diterima dalam urutan yang benar. Server kemudian mengirimkan balasan dan menutup koneksi socket. Di sisi client, socket dibuat dengan alamat host dan port yang sesuai, mengirimkan request ke server, kemudian menunggu dan menerima balasan

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

dari server, sebelum akhirnya menutup koneksi socket tersebut. Protokol TCP memastikan pengiriman data yang teratur dan dapat diandalkan melalui proses koneksi yang lebih kompleks.

4. Jelaskan struktur alamat yang digunakan dalam socket programming. Apa yang dimaksud dengan IP Address dan Port Number, serta bagaimana keduanya bekerja bersama untuk menentukan tujuan komunikasi?

Socket address terdiri dari kombinasi IP Address dan Port Number yang bekerja bersama untuk memungkinkan komunikasi yang tepat antar perangkat dalam sebuah jaringan. IP Address adalah alamat numerik yang berfungsi untuk mengidentifikasi perangkat dalam jaringan. Untuk IPv4, alamat ini menggunakan format 32-bit, sementara untuk IPv6, alamatnya menggunakan format 128-bit. Setiap perangkat yang terhubung ke jaringan memiliki IP Address yang unik, yang memungkinkan perangkat lain untuk menemukan dan berkomunikasi dengannya. Alamat ini digunakan untuk memastikan bahwa data yang dikirim dapat sampai ke perangkat yang tepat dalam jaringan.

Port Number, di sisi lain, adalah angka 16-bit yang berkisar antara 0 hingga 65535, yang digunakan untuk mengidentifikasi aplikasi atau proses tertentu yang berjalan pada perangkat tersebut. Port Number bekerja bersama IP Address untuk menunjukkan aplikasi yang tepat pada perangkat yang terhubung ke jaringan. Ketika sebuah perangkat mengirimkan data ke IP Address tertentu, Port Number memastikan bahwa data tersebut diteruskan ke aplikasi yang sesuai pada perangkat tersebut. Kombinasi IP Address dan Port Number, yang dikenal sebagai socket address, memungkinkan aplikasi untuk saling berkomunikasi dengan tepat dan efisien melalui jaringan.

5. Dalam socket programming akan ada kasus yang akan memerlukan Thread yang sudah anda pelajari pada Mata Kuliah Sistem Operasi. Sehingga jelaskan apa yang dimaksud dengan Threading dan Multi-Threading! Selain itu pada code dibawah ini mana yang termasuk Threading dan Multi-Threading? Berikan penjelasannya!

Threading adalah unit eksekusi terkecil dalam sebuah proses yang memungkinkan beberapa thread berjalan secara konkuren dalam satu proses, berbagi memori dan sumber daya yang sama. Setiap thread memiliki stack sendiri namun berbagi heap dengan thread lainnya dalam proses yang sama, dan digunakan untuk menjalankan tugas-tugas kecil secara paralel. Sementara itu, Multi-Threading adalah teknik yang memungkinkan aplikasi menjalankan beberapa thread secara bersamaan dalam satu proses untuk meningkatkan efisiensi. Teknik ini mengoptimalkan penggunaan sumber daya perangkat seperti CPU, GPU, dan memori, serta meningkatkan responsivitas perangkat dengan menjaga bagian-bagian penting, seperti antarmuka pengguna (GUI), tetap responsif meskipun tugas berat berjalan di latar belakang. Multi-threading juga memungkinkan penanganan banyak koneksi atau tugas secara paralel, yang meningkatkan kinerja dan responsivitas aplikasi.

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

- **Code 1**

```
import threading

def task():
    print("Task is running")

thread = threading.Thread(target=task)
thread.start()
thread.join()
```

Code 1 dapat dikategorikan sebagai threading karena hanya ada satu thread yang dibuat, yaitu `thread = threading.Thread(target=task)`. Saat dieksekusi, main thread membuat satu thread baru untuk menjalankan fungsi `task()`, dan main thread akan terhenti atau diblokir pada `thread.join()` hingga thread selesai. Ini tidak dapat disebut sebagai multi-threading karena hanya ada satu thread tambahan yang berjalan bersamaan dengan main thread, bukan beberapa thread yang berjalan paralel.

- **Code 2**

```
import threading

def task(name):
    print(f"Task {name} is running")

threads = []

for i in range(5):
    thread = threading.Thread(target=task, args=(i,))
    threads.append(thread)
    thread.start()

for thread in threads:
    thread.join()
```

Code 2 termasuk dalam kategori Multi-Threading karena melibatkan banyak thread yang bekerja secara bersamaan. Dalam hal ini, lima thread dibuat sekaligus melalui perulangan `for i in range(5)`. Ketika dieksekusi, kelima thread tersebut berjalan secara paralel (concurrent), sementara main thread menunggu hingga semua thread selesai dieksekusi dengan cara memblokir di `join()`.

6. Dalam network programming, sering kali terjadi error seperti timeout, connection refused, atau address already in use. Jelaskan penyebab umum dari masing-masing error berikut dan strategi umum untuk menangannya:

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

a. Timeout

Timeout dapat terjadi karena beberapa faktor, seperti jaringan yang lambat yang menyebabkan keterlambatan dalam merespon, server yang tidak merespon permintaan dari client atau sebaliknya, adanya firewall atau pemblokiran jaringan pada server atau client, atau konfigurasi timeout yang terlalu singkat dalam sistem. Untuk menangani masalah ini, beberapa strategi dapat diterapkan, antara lain meningkatkan waktu deteksi timeout, menggunakan mekanisme retry (retransmission) untuk mengulang permintaan yang gagal, dan menggunakan fallback server, yang memungkinkan perpindahan ke server lain saat terjadi timeout, guna menjaga kelancaran komunikasi dan mengurangi dampak dari masalah tersebut.

b. Connection Refused

Connection Refused terjadi ketika server tidak berjalan dan client gagal menghubungi server, atau ketika port pada client atau server tidak terbuka, alamat IP yang digunakan tidak sesuai atau salah, atau penggunaan protokol yang tidak cocok. Untuk menangani masalah ini, beberapa strategi yang dapat diterapkan meliputi memeriksa status server untuk memastikan server aktif, memverifikasi port dan alamat IP yang benar untuk server dan client, mengonfigurasi firewall dengan benar agar tidak memblokir koneksi yang sah, serta melakukan mekanisme error handling untuk menangani kesalahan dengan lebih efektif dan memberikan respons yang tepat kepada pengguna atau sistem yang terkait.

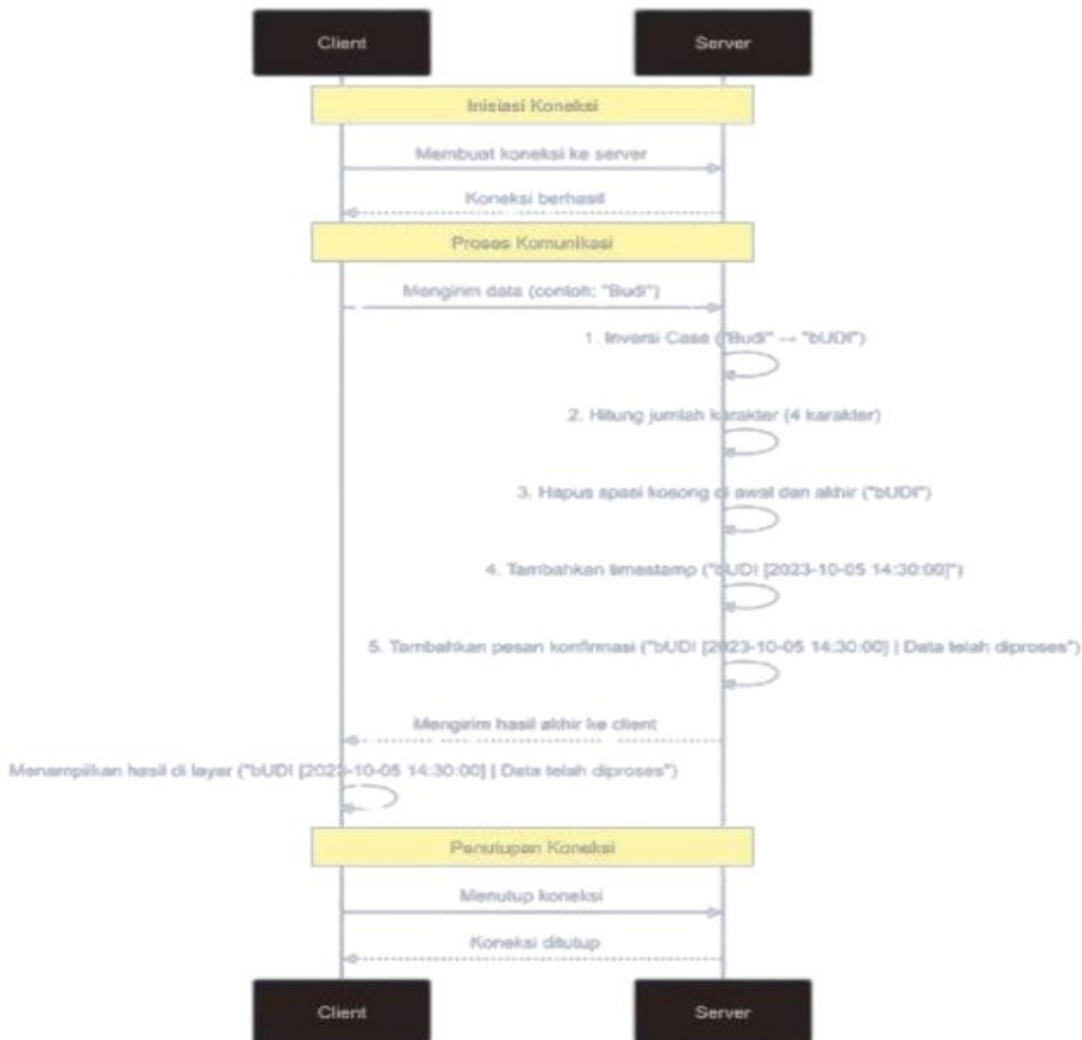
c. Address Already in Use

Address Already in Use terjadi ketika port yang digunakan sudah dipakai oleh perangkat atau aplikasi lain, atau ketika socket tidak dibebaskan (ditutup dan dibuka kembali untuk digunakan oleh perangkat atau aplikasi lain), atau ketika port berada dalam fase TIME_WAIT state, yaitu ketika TCP menahan port sementara setelah koneksi ditutup. Untuk menangani masalah ini, beberapa strategi yang dapat diterapkan antara lain menggunakan port lain yang tidak sedang digunakan, mengonfigurasi socket untuk memungkinkan reuse address, menunggu hingga proses yang menggunakan port tersebut selesai atau membunuh proses yang mengunci port, serta menghindari penggunaan TIME_WAIT untuk meningkatkan responsivitas socket atau port address, sehingga port dapat digunakan kembali dengan lebih cepat.

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

Berikut merupakan algoritma sederhana untuk socket programming yang akan digunakan pada poin B dan poin C: Program ini adalah program client-server sederhana. Client mengirimkan data ke server, server melakukan 5 proses berbeda terhadap data tersebut, lalu mengembalikan hasilnya ke client untuk ditampilkan.



Langkah-Langkah Algoritma

1. Inisialisasi Server:

- Server membuat socket untuk mendengarkan koneksi masuk dari client.
- Server mengikat socket ke alamat IP dan port tertentu.
- Server mulai mendengarkan koneksi masuk dari client.

2. Inisialisasi Client:

- Client membuat socket untuk terhubung ke server.
- Client menghubungkan socket ke alamat IP dan port server.

JARINGAN KOMPUTER
MODUL 7 SOCKET PROGRAMMING

3. Proses Komunikasi:

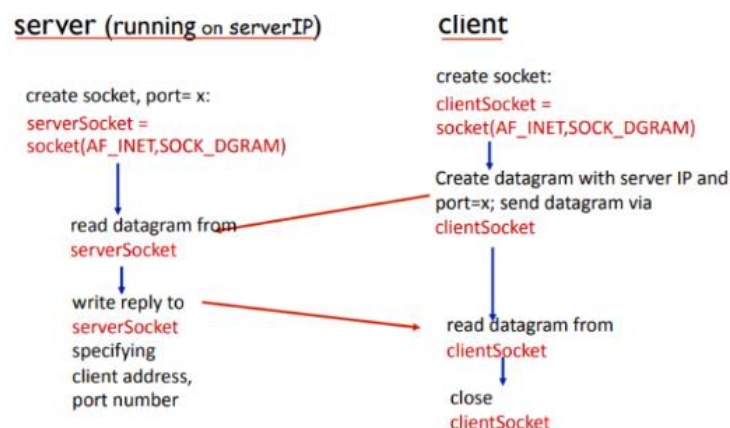
- Client: a. Membaca sebaris karakter (data) dari input pengguna (keyboard).
b. Mengirimkan data tersebut ke server melalui socket.
- Server: a. Menerima data dari client melalui socket.
b. Melakukan 5 proses berikut terhadap data:
 - 1) Inversi Case: Jika karakter adalah huruf kecil, ubah menjadi kapital; jika kapital, ubah menjadi kecil. Karakter non-huruf tetap tidak berubah.
 - 2) Menghitung jumlah karakter dalam data.
 - 3) Menghapus semua spasi kosong di awal dan akhir data.
 - 4) Menambahkan timestamp (waktu saat ini) ke data.
 - 5) Menambahkan pesan konfirmasi "Data telah diproses".
c. Mengirimkan data yang telah dimodifikasi kembali ke client.
- Client: a. Menerima data yang telah dimodifikasi dari server.
b. Menampilkan data tersebut di layar.

4. Penutupan Koneksi:

- Setelah selesai, baik client maupun server menutup socket untuk mengakhiri komunikasi.

B. Socket Programming with UDP

1. Buatlah program client dan server dengan UDP menggunakan bahasa pemrograman Python sesuai dengan algoritma diatas! Untuk mempermudah pemahaan berkaitan algoritma diatas berikut skema program nanti harus berjalan!



Berikan nama setiap file dengan UDPClient_NIM.py dan UDPServer.py!

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

```
UDPServer.py
1  import socket
2  import datetime
3
4  def invert_case(text):
5      # Menukar Huruf
6      return text.swapcase()
7
8  def count_characters(text):
9      # Menghitung Jumlah huruf
10     return len(text)
11
12     def trim_spaces(text):
13         # Menghapus space pada awal dan akhir
14         return text.strip()
15
16     def add_timestamp(text):
17         # Menambah Timestamp
18         timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
19         return f"{text} {timestamp}"
20
21     def add_confirmation(text):
22         # Menambah Konfirmasi
23         return f"{text} | Data telah diproses"
24
25     def process_data(data):
26         # Memberikan 5 step pada pesan yang diterima dari client
27         # Step 1: Tukar huruf
28         processed = invert_case(data)
29
30         # Step 2: Jumlah characters
31         char_count = count_characters(data)
32
33         # Step 3: Hapus space pada awal dan akhir
34         processed = trim_spaces(processed)
35
36         # Step 4: Menambah timestamp
37         processed = add_timestamp(processed)
38
39         # Step 5: Menambah Konfirmasi
40         processed = add_confirmation(processed)
41
42         return f"{processed} (Jumlah: {char_count} Characters)"
43
44     def start_server():
45         # Membuat UDP Socket Server
46         server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
47
48         # Bind UDP Socket pada Localhost dengan port 80
49         server_address = ('localhost', 80)
50         server_socket.bind(server_address)
51         print(f"Server sekarang terbuka di address: {server_address[0]}:{server_address[1]}")
52
53         try:
54             print("\nMenunggu data dari client...")
55
56             # Menerima data dari client
57             data, client_address = server_socket.recvfrom(4096)
58
59             print(f"Menerima {len(data)} bytes dari {client_address}")
60             decoded_data = data.decode('utf-8')
61             print(f"Data Diterima: {decoded_data}")
62
63             # Memproses Data dari client di server
64             print("Melakukan Proses Data dari Client di Server")
65             processed_data = process_data(decoded_data)
66             print(f"Hasil Proses Data: {processed_data}")
67
68             # Mengirim Data yang telah diproses di server
69             server_socket.sendto(processed_data.encode('utf-8'), client_address)
70             print(f"Mengirim data kembali ke client dengan address: {client_address}")
71
72         finally:
73             # Menutup UDP connection pada server
74             server_socket.close()
75             print("Connection Terputus")
76
77     if __name__ == "__main__":
78         start_server()
79
```

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

```
UDPclient_103032300013.py
1  import socket
2
3  def start_client():
4      # Create a UDP socket
5      client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6
7      # Server address
8      server_address = ('localhost', 80)
9
10     try:
11         # Get input from user
12         message = 'Budi'
13
14         # Send data to server
15         print(f"Mengirim data dengan pesan: {message}")
16         client_socket.sendto(message.encode('utf-8'), server_address)
17
18         # Receive response from server
19         print("Menunggu data / respon dari server...")
20         response, _ = client_socket.recvfrom(4096)
21         print(f"Menerima data / respon dari server: {response.decode('utf-8')}")
22
23     finally:
24         client_socket.close()
25         print("Connection Terputus")
26
27 if __name__ == "__main__":
28     print("Menjalankan Client. Menghubungkan ke address localhost:80")
29     start_client()
30
```

2. Eksekusi code yang sudah dibuat sehingga dapat menunjukkan hubungan antara client-server (WAJIB GUNAKAN CMD/TERMINAL)! Jika tidak muncul hubungan tersebut berarti ada masalah dengan code yang anda buat, silahkan diperbaiki!

```
C:\Exercise Program\SOCKET PROGRAMMING>py UDPserver.py
Server sekarang terbuka di address: localhost:80

Menunggu data dari client...
Menerima 4 bytes dari ('127.0.0.1', 50402)
Data Diterima: Budi
Melakukan Proses Data dari Client di Server
Hasil Proses Data: bUDI 2025-04-08 00:00:59 | Data telah diproses (Jumlah: 4 Characters)
Mengirim data kembali ke client dengan address: ('127.0.0.1', 50402)
Connection Terputus
```

```
C:\Exercise Program\SOCKET PROGRAMMING>py UDPclient_103032300013.py
Menjalankan Client. Menghubungkan ke address localhost:80
Mengirim data dengan pesan: Budi
Menunggu data / respon dari server...
Menerima data / respon dari server: bUDI 2025-04-08 00:00:59 | Data telah diproses (Jumlah: 4 Characters)
Connection Terputus
```

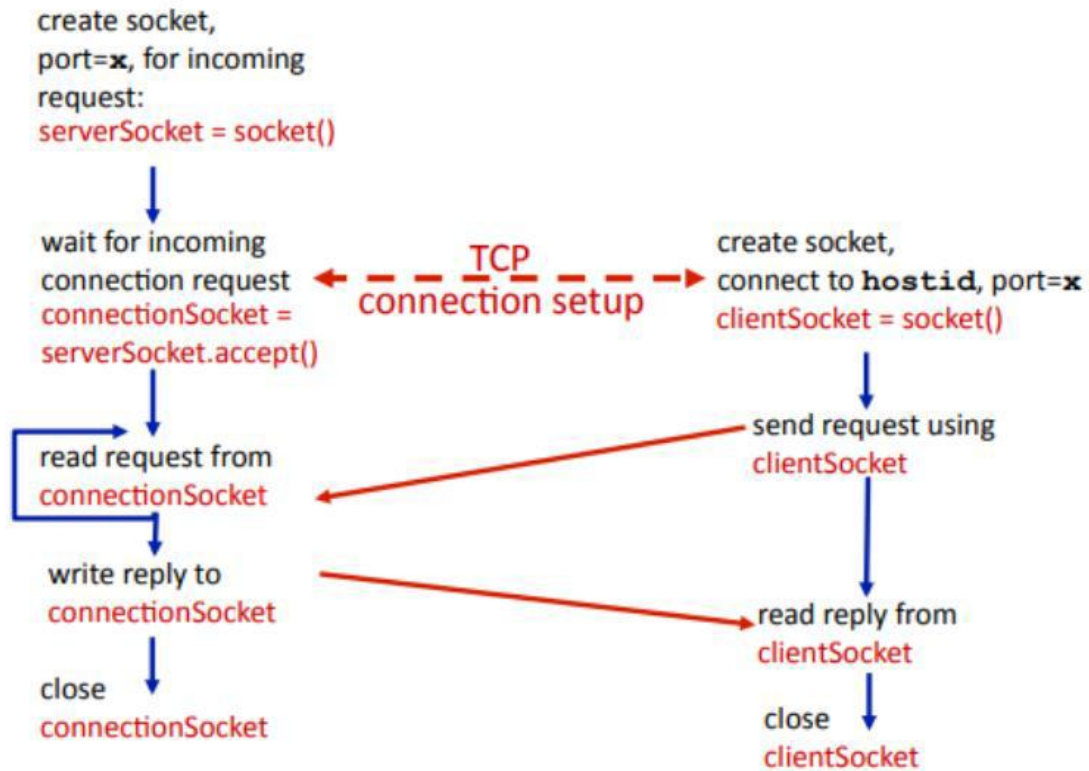
C. Socket Programming with TCP

1. Buatlah program client dan server dengan TCP menggunakan bahasa pemrograman Python sesuai dengan algoritma diatas! Untuk mempermudah pemahaman berkaitan algoritma diatas berikut skema program nanti harus berjalan

JARINGAN KOMPUTER
MODUL 7 SOCKET PROGRAMMING

server (running on hostid)

client



Berikan nama setiap file dengan TCPClient_NIM.py dan TCPServer.py!

```
TCPServer.py
1 import socket
2 import datetime
3
4 def invert_case(text):
5     # Menukar Huruf
6     return text.swapcase()
7
8 def count_characters(text):
9     # Menghitung Jumlah huruf
10    return len(text)
11
12 def trim_spaces(text):
13    # Menghapus space pada awal dan akhir
14    return text.strip()
15
16 def add_timestamp(text):
17    # Menambah Timestamp
18    timestamp = datetime.datetime.now().strftime("[%Y-%m-%d %H:%M:%S]")
19    return f"{text} {timestamp}"
20
21 def add_confirmation(text):
22    # Menambah Konfirmasi
23    return f"{text} | Data telah diproses"
24
25 def process_data(data):
26    # Memberikan 5 step pada pesan yang diterima dari client
27    # Step 1: Tukar huruf
28    processed = invert_case(data)
29
30    # Step 2: Jumlah characters
31    char_count = count_characters(data)
32
```

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

```
33     # Step 3: Hapus space pada awal dan akhir
34     processed = trim_spaces(processed)
35
36     # Step 4: Menambah timestamp
37     processed = add_timestamp(processed)
38
39     # Step 5: Menambah Konfirmasi
40     processed = add_confirmation(processed)
41
42     return f"{processed} (Jumlah: {char_count} Characters)"
43
44 def start_server():
45     # Membuat TCP Socket Server
46     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
47
48     # Bind TCP Socket pada Localhost dengan port 8080
49     server_address = ('localhost', 8080)
50     server_socket.bind(server_address)
51     server_socket.listen(1) # Maksimal 1 koneksi dalam antrian
52     print(f"Server sekarang terbuka di address: {server_address[0]}:{server_address[1]}")
53
54     try:
55         print("\nMenunggu koneksi dari client...")
56
57         # Menerima koneksi dari client
58         connection, client_address = server_socket.accept()
59         print(f"Koneksi diterima dari {client_address}")
60
61         try:
62             # Menerima data dari client
63             data = connection.recv(4096)
64             print(f"Menerima {len(data)} bytes dari {client_address}")
65             decoded_data = data.decode('utf-8')
66             print(f"Data Diterima: {decoded_data}")
67
68             # Memproses Data dari client di server
69             print("Melakukan Proses Data dari Client di Server")
70             processed_data = process_data(decoded_data)
71             print(f"Hasil Proses Data: {processed_data}")
72
73             # Mengirim Data yang telah diproses di server
74             connection.sendall(processed_data.encode('utf-8'))
75             print(f"Mengirim data kembali ke client")
76
77         finally:
78             # Menutup koneksi TCP
79             connection.close()
80             print(f"Koneksi dengan {client_address} ditutup")
81
82     finally:
83         # Menutup socket server
84         server_socket.close()
85         print("Server ditutup")
86
87 if __name__ == "__main__":
88     start_server()
89
```

JARINGAN KOMPUTER MODUL 7 SOCKET PROGRAMMING

```
TCPCClient_103032300013.py
1  import socket
2
3  def start_client():
4      client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6      server_address = ('localhost', 8080)
7
8      try:
9          print(f"Menghubungkan ke server {server_address[0]}:{server_address[1]}...")
10         client_socket.connect(server_address)
11         print("Berhasil terhubung ke server!")
12
13         message = 'Budi'
14
15         print(f"Mengirim data dengan pesan: {message}")
16         client_socket.sendall(message.encode('utf-8'))
17
18         print("Menunggu data respon dari server...")
19         response = client_socket.recv(4096)
20         print(f"Menerima data / respon dari server: {response.decode('utf-8')}")
21
22     except ConnectionRefusedError:
23         print("Tidak ada respon dari server")
24
25     finally:
26         client_socket.close()
27         print("Koneksi ditutup")
28
29 if __name__ == "__main__":
30     print("Menjalankan Client TCP. Menghubungkan ke address localhost:8080")
31     start_client()
```

2. Eksekusi code yang sudah dibuat sehingga dapat menunjukkan hubungan antara client-server (WAJIB GUNAKAN CMD/TERMINAL)! Jika tidak muncul hubungan tersebut berarti ada masalah dengan code yang anda buat, silahkan diperbaiki!

```
C:\Exercise Program\Socket Programming with TCP>py TCPserver.py
Server sekarang terbuka di address: localhost:8080

Menunggu koneksi dari client...
Koneksi diterima dari ('127.0.0.1', 51074)
Menerima 4 bytes dari ('127.0.0.1', 51074)
Data Diterima: Budi
Melakukan Proses Data dari Client di Server
Hasil Proses Data: bUDI [2025-04-08 00:20:17] | Data telah diproses (Jumlah: 4 Characters)
Mengirim data kembali ke client
Koneksi dengan ('127.0.0.1', 51074) ditutup
Server ditutup
```

```
C:\Exercise Program\Socket Programming with TCP>py TCPCClient_103032300013.py
Menjalankan Client TCP. Menghubungkan ke address localhost:8080
Menghubungkan ke server localhost:8080...
Berhasil terhubung ke server!
Mengirim data dengan pesan: Budi
Menunggu data respon dari server...
Menerima data / respon dari server: bUDI [2025-04-08 00:20:17] | Data telah diproses (Jumlah: 4 Characters)
Koneksi ditutup
```

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

D. Analisa Socket Program

Pada tahap ini akan melakukan analisa pada kode yang berikan serta menjawab pertanyaan-pertanyaan yang diberikan:

```
import socket

def resolve_dns(domain_name):
    dns_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    dns_server = "8.8.8.8"
    dns_port = 53

    dns_query = b"\x00\x01\x01\x00\x00\x01\x00\x00\x00\x00\x00" + \
        domain_name.encode('utf-8') + b"\x00\x00\x01\x00\x01"

    try:
        print(f"Mengirim permintaan DNS untuk {domain_name}...")
        dns_socket.sendto(dns_query, (dns_server, dns_port))
        dns_socket.settimeout(5) # Timeout 5 detik

        response, _ = dns_socket.recvfrom(1024)
        print("Respon DNS diterima.")

        ip_address = socket.inet_ntoa(response[-4:])
        print(f"Alamat IP untuk {domain_name}: {ip_address}")
        return ip_address
    except socket.timeout:
        print("Timeout: Tidak ada respon dari server DNS.")
        return None
    finally:
        dns_socket.close()

def fetch_web_page(ip_address):
    http_request = f"GET / HTTP/1.1\r\nHost: {ip_address}\r\nConnection: close\r\n\r\n"

    tcp_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    try:
        print(f"Menghubungkan ke server web di {ip_address}:80...")
        tcp_socket.connect((ip_address, 80))
        tcp_socket.sendall(http_request.encode('utf-8'))

        response = b""
        while True:
            data = tcp_socket.recv(4096)
            if not data:
                break
            response += data

        print("Respon HTTP diterima.")
        return response.decode('utf-8')
    except Exception as e:
        print(f"Error: {e}")
        return None
    finally:
        tcp_socket.close()

# Main program
if __name__ == "__main__":
    domain_name = "example.com"

    ip_address = resolve_dns(domain_name)
    if ip_address:
        print("\nMengunduh halaman web...")
        web_content = fetch_web_page(ip_address)
        if web_content:
            print("\nKonten Halaman Web:")
            print(web_content[:500])
        else:
            print("Gagal mengunduh halaman web.")
    else:
        print("Gagal mendapatkan alamat IP dari DNS.")
```

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

1. Apa maksud kode diatas?

Berfungsi sebagai DNS resolver yang mengonversi nama domain, seperti "domain.com", menjadi alamat IP yang sesuai. Setelah DNS berhasil diresolve dan alamat IP ditemukan, aplikasi kemudian menggunakan IP tersebut untuk melakukan fetch pada website yang terkait. Proses ini dimulai dengan memasukkan DNS ke dalam resolver, yang akan mengembalikan alamat IP yang relevan. Dalam konfigurasi aplikasi, kita menentukan jenis protokol yang digunakan, menetapkan timeout untuk koneksi, dan menyiapkan prosedur untuk menangani berbagai kondisi, seperti error handling ketika IP tidak ditemukan atau kesalahan lainnya yang terjadi selama proses fetch website. Dengan begitu, aplikasi dapat memberikan respons yang tepat ketika terjadi kegagalan atau kesalahan dalam mencari alamat IP atau mengakses website.

2. Protokol apa saja yang akan digunakan pada kode diatas?

Penggunaan DNS resolve dengan UDP dapat dilihat melalui tipe socket `SOCK_DGRAM`, yang memungkinkan komunikasi tanpa koneksi (connectionless) untuk mengirimkan permintaan DNS dan menerima respons. Sementara itu, untuk melakukan koneksi dan mengirimkan permintaan fetch website setelah DNS berhasil diresolve, protokol TCP digunakan dengan tipe socket `SOCK_STREAM`. Protokol TCP ini memastikan adanya koneksi yang stabil antara client dan server, memungkinkan permintaan resolusi DNS dilakukan dengan aman, dan setelah terhubung, aplikasi dapat melanjutkan untuk mengambil data dari website yang dituju sesuai dengan konfigurasi yang telah ditetapkan, seperti timeout dan pengaturan koneksi lainnya.

3. Bagaimana program ini menggunakan protokol TCP untuk mengunduh halaman web melalui HTTP? Jelaskan langkah-langkah yang dilakukan oleh program untuk membuat koneksi TCP dan mengirim permintaan HTTP. Mengapa protokol TCP lebih cocok daripada UDP untuk HTTP?

Untuk melakukan HTTP fetching menggunakan TCP, langkah pertama adalah membuat socket TCP dengan tipe socket `AF_INET` dan `SOCK_STREAM`, yang menggunakan protokol TCP. Setelah socket dibuat, langkah berikutnya adalah menghubungkan ke server menggunakan alamat IP dan nomor port default 80. Setelah terhubung, aplikasi mengirimkan permintaan HTTP yang telah diproses, termasuk domain name atau IP address, dan kemudian menunggu respon dari server terkait request HTTP atau DNS. Respons tersebut diterima menggunakan `tcp_socket.recv(4096)` dengan ukuran buffer default, dan akhirnya, koneksi ditutup menggunakan perintah `tcp_socket.close()`, yang mengirimkan flag seperti FIN dan ACK untuk mengakhiri sesi.

TCP lebih cocok untuk HTTP dibanding UDP karena memberikan keandalan (reliability) yang memastikan seluruh data atau konten dapat diterima dengan benar, serta mekanisme retry ketika terjadi gangguan. Selain itu, TCP menjamin urutan data yang tepat, yang sangat penting dalam HTTP untuk menjaga struktur yang sesuai, seperti pada HTML, CSS, dan JavaScript yang umumnya terstruktur secara terurut. TCP juga mendukung ukuran transfer data yang besar, yang diperlukan dalam pengambilan webpage dengan data besar, dan memiliki aliran kontrol (flow control) untuk

JARINGAN KOMPUTER

MODUL 7 SOCKET PROGRAMMING

menghindari overload pada client atau server. Terakhir, TCP cocok untuk koneksi yang membutuhkan stateful, yang mana HTTP sering kali membutuhkan sesi multi-request untuk memastikan integritas data dan kelancaran transfer informasi.

4. Jelaskan bagaimana program ini menggunakan protokol UDP untuk menyelesaikan nama domain (example.com) menjadi alamat IP. Mengapa protokol UDP dipilih untuk komunikasi DNS? Apa keuntungan dan kerugian penggunaan UDP dalam konteks ini?

Program ini menggunakan protokol UDP untuk menyelesaikan nama domain (misalnya example.com) menjadi alamat IP dengan cara mengonfigurasi aplikasi untuk menggunakan `dns_server`, tipe `dns_socket`, dan `dns_port` yang diperlukan untuk proses resolve DNS. Setelah konfigurasi selesai, program akan melakukan `dns_query` yang mengubah permintaan menjadi bentuk hexadecimal untuk melakukan query DNS. Selanjutnya, permintaan dikirim ke server DNS menggunakan perintah `dns_socket.sendto(dns_query, (dns_server, dns_port))`. Program juga mengonfigurasi waktu timeout untuk maksimal 5 detik menggunakan `dns_socket.settimeout(5)`. Setelah itu, program akan menunggu respon dari server DNS menggunakan `dns_socket.recv()`. Jika respon diterima dalam waktu yang ditentukan, IP address akan diperoleh menggunakan `socket.inet_ntoa(response[-4:])`, dan nilai IP yang telah diresolve akan dikembalikan. Jika tidak ada respon dalam waktu 5 detik, akan terjadi timeout dan program akan mengembalikan error.

Protokol UDP dipilih untuk komunikasi DNS karena memiliki overhead yang rendah, hanya 8 byte, dibandingkan dengan TCP yang membutuhkan 20 byte overhead, ditambah dengan prosedur handshake yang lebih rumit. UDP sangat cocok untuk DNS karena digunakan untuk resolusi DNS berukuran kecil dan memiliki kecepatan tinggi, mengingat UDP tidak memerlukan prosedur handshake seperti TCP. Selain itu, DNS query sering kali hanya memerlukan satu permintaan dan satu respons, yang sesuai dengan prinsip stateless dari UDP. Server DNS tidak perlu menyimpan status koneksi, sehingga memudahkan load balancing dan scaling, serta memungkinkan DNS menangani banyak query dalam waktu singkat dengan efisien.

Keuntungan penggunaan UDP untuk DNS meliputi latensi rendah karena tidak adanya prosedur handshake atau acknowledgment, implementasi protokol yang sederhana, dan efisiensi karena query DNS biasanya kecil dan diselesaikan dalam satu paket (single request-reply). UDP juga mendukung scalability, di mana server DNS dapat menangani lebih banyak query secara bersamaan tanpa terhambat oleh keterbatasan koneksi. Kerugian dari penggunaan UDP dalam DNS termasuk ketidakandalan (unreliable), karena paket dapat hilang, sehingga perlu ada penanganan error seperti retry atau retransmission. Selain itu, UDP tidak memiliki flow control, yang dapat menyebabkan masalah jika aliran data yang datang terlalu banyak dan tidak dapat dikendalikan. Ukuran respons DNS juga terbatas hingga 512 byte, yang membatasi jumlah data yang dapat dikirimkan dalam satu kali permintaan. Terakhir, risiko spoofing lebih tinggi dengan UDP, karena tidak menggunakan sequence number untuk verifikasi dan sinkronisasi antar paket, sehingga lebih rentan terhadap manipulasi data.

JARINGAN KOMPUTER
MODUL 7 SOCKET PROGRAMMING

5. Berikan kemungkinan output dari kode tersebut

Jika menggunakan kode langsung dari soal, maka kemungkinan besar situs tidak dapat diakses karena terjadi timeout dan tidak ada respons dari server DNS, sehingga alamat IP tidak dapat diperoleh. Hal ini bisa disebabkan oleh informasi yang tidak lengkap atau struktur yang tidak sesuai pada DNS query yang dikirim, sehingga server DNS gagal memberikan respon yang benar.